RFC-001: A Uniform, Modular Sensor Data Model

Status: Draft, open for comment **Author:** Calvin Grabowski **Date:** October 22, 2026

Project: DIAL Control Center (OSU mobile daylighting research trailer)

Decision type: Mixed. The metric names and units we store are a Type 1 decision (hard to undo later). The driver code around them is a Type 2 decision (easy to change later). These terms come from Amazon's 2015 shareholder letter [6].

1. Problem and Context

The Control Center saves every instrument reading in a single SQLite database file. It currently works with three sensor kinds: **T-10A illuminance meters**, **JETI Spectraval spectroradiometers**, and an **EKO MS-90+ solar station**.

Sensor data lives in four tables (svc/app/state.py, _ensure_sensor_db): **sensors** (id, kind, label, location, config_json, active); **sensor_readings** (sensor, timestamp, metric, value); **sensor_spectra** (spectra as JSON arrays); and **sensor_ingest_events** (idempotency for the Windows Sensor Agent).

The sensor_readings table is already sensor-agnostic by being one row per measurement, no new columns needed for a new instrument. The problems surround it:

- **Metric names follow no rules.** Each driver picks its own: the T-10A writes lux; the JETI spectral pipeline writes 22 metrics such as melanopic_edi_lx and cct_ohno_k; the EKO writes ghi_w_m2, board_temp_c, and gps_satellites. The database records neither unit nor quantity. The only arrival check is a name pattern (MetricName in models.py), so a driver writing illuminance instead of lux would produce a silently different metric.
- **Adding a sensor kind means editing shared core code.** Two functions in svc/app/sensors/manager.py name each of the three sensor kinds explicitly and contain a separate block per kind, with a hard limit of four devices each. A fourth kind means editing a 1,144-line file every sensor depends on.
- **Schema changes are ad hoc.** The active column was added by checking PRAGMA table_info and running ALTER TABLE. Nothing records which version of the structure a database file has, so there is no reliable way to tell which schema it follows.

Why this is hard to reverse. Driver registration code is easy to change. The stored metric vocabulary is not: once a study records readings under a name, renaming it means migrating

historical rows, otherwise earlier results are incomparable. **Why now:** before more instruments are added and before this year's study data accumulates under names nobody chose deliberately. If we do nothing, each new instrument adds names by its author's habit, units stay undocumented, and every addition risks breaking existing drivers.

2. Options

A. Do nothing. Keep free-form metric names and per-kind branches. Cost: nothing now, but each new sensor gets harder to add and the research data vocabulary drifts.

B. Keep the long table; add a metric catalog, a driver registry, and schema versioning. A new metric_definitions table records each metric's name, quantity, unit, and description. Each driver declares the metrics it emits. A registry maps each config key to its validator and client factories, so the core no longer needs a block per kind. PRAGMA user_version tracks migrations. Cost: one migration, a refactor of manager.py, and seeding the catalog with the 34 metrics produced today (1 from T-10A, 11 from EKO, 22 from JETI).

C. One wide table per sensor kind (e.g. t10a_readings(lux), eko_readings(ghi_w_m2, ...)). Cost: every new sensor or metric is a schema migration, and the queries behind the existing API become UNIONs across tables that grow with each sensor.

D. Move readings to a time-series database such as InfluxDB or TimescaleDB. Cost: a second service on the trailer computer, a new startup failure mode, migrating existing data and the db_backup.py workflow, and a license review.

3. Analysis

Performance. The database already has an index on (sensor_id, metric, ts) that keeps range and latest-value queries fast. Option B leaves it untouched. SQLite's own guidance describes local, single-application, low-to-moderate-write workloads like ours as a good fit [3]. Section 4 specifies how performance will be measured.

Maintainability. Option B keeps each instrument's knowledge in its own driver file and catalog entries, matching the SensorClient protocol boundary in interface.py (a structural Protocol per PEP 544 [5]). Option C puts that knowledge into the schema, where SQLite's limited ALTER TABLE support makes changes costly [4].

Precedent for uniformity. IETF RFC 8428 (SenML) represents each measurement as name, unit, value, and time, with units from a registry such as lx, W/m², and Cel [1]. Option B adopts that shape internally.

Security. Only option D adds attack surface: a network service and its credentials.

4. Recommendation and Verification Plan

Adopt option B. Concretely:

1. Add schema versioning with PRAGMA user_version [2] and ordered migration functions in state.py. Version 1 is today's schema; version 2 adds metric_definitions (name PRIMARY KEY, quantity, unit, description, status).
2. Seed the catalog with every metric current driver and simulator emit. Existing names are kept exactly, so history and API responses are unchanged. New names follow <quantity>_<qualifier>_<unit> with SenML unit symbols where one exists.
3. Unknown metrics are still accepted (rejecting them would change ingest status-code behavior) but are recorded with status = 'unreviewed' and logged, so they are visible rather than silent.
4. Replace per-kind blocks in manager.py with a registry: config key to validator, physical factory, simulator factory, and declared metrics. Adding a sensor means a new driver module, a simulator, a registry entry, and catalog rows. Removing one deletes its entry; history is kept via the existing inactive flag.

How we will know it was right:

- **Migration test:** a version-1 database built from simulator data upgrades to version 2 with identical row counts, and the sensor list, latest readings, log, and spectrum endpoints return byte-identical JSON before and after.
- **Contract test:** every metric, any driver or simulator can emit exists in the catalog with a non-empty unit. This check runs in CI (.github/workflows/ci.yml) and fails the moment a driver uses an unregistered name.
- **Modularity acceptance test:** a test-only sensor kind is added by creating one new module and one registry entry; the diff to manager.py is zero lines and all existing tests pass.
- **Failure paths:** with an instrument missing or unreachable, startup still succeeds and health reports it as degraded; an unknown metric is stored and flagged, never crashes ingestion.
- **Performance:** on a database seeded with one simulated month of all-sensor data, p95 time for a 1,000-event ingest batch and for the latest-readings query must not regress more than 10% from the version-1 baseline.
- **Signal to reconsider:** if either of the next two sensor additions needs a kind-specific branch in manager.py or bypasses the catalog, this RFC is reopened.

5. Constraints

- **API contract:** AGENTS.md prohibits adding, removing, renaming, or altering routes, methods, request/response schemas, or status-code behavior; SensorIngestEvent, SensorReadingResponse, and SensorLogEntry must not change. Everything proposed here is additive and internal. If any part requires an API change, work stops and the issue is raised with the team.
- **Licensing:** the repository is GPL-3.0. Option B adds no new dependencies. SenML is a design reference only. Option D would require a license review.
- **Data use:** readings are research records. This RFC and its tests use only the simulator and committed fixtures; no raw trailer data is used or shared.
- **Partner and vendor confidentiality:** vendor protocol details (Modbus register maps, Halio API material) are not reproduced here. The change sits above the driver layer and does not bypass vendor software, so calibration is unaffected.
- **AI tool approval:** AGENTS.md notes that no approved-tool list or data classification is recorded yet. Only public project code was shared with an AI tool for this draft.

6. Delegation and Validation

Handed to AI (Claude, via claude.ai): reading state.py, models.py, manager.py, and the driver modules to summarize the current schema and metric names; drafting the option descriptions and prose; proposing the verification tests.

Checks that make that reasonable: every code claim names a file a reviewer can open; references are primary sources checked against their URLs; no benchmarks are claimed; implementation goes through CI, pull-request review, and a restorable database backup as the rollback path.

Where I stopped and validated myself:

1. Opened manager.py and confirmed the three hardcoded kind blocks (T-10A, JETI, EKO), the 4-device cap in _require_list(), and the actual line count of 1,144.
2. Opened AGENTS.md and confirmed the API constraint language: routes, methods, schemas, and status-code behavior must not change.
3. Opened t10a_client.py, eko_cbox_modbus_tcp_client.py, jeti_spectraval_watcher.py, and spectral_metrics.py and counted every metric each driver emits: 1 (T-10A), 11 (EKO), 22 (JETI) = 34 total. The RFC metric list matches the code.
4. Still to do before implementation: confirm with the project partner which metric names and units researchers use in their analysis scripts and run the migration on a copy of a backup before it touches production data.

References

[1] C. Jennings, Z. Shelby, J. Arkko, A. Keranen, C. Bormann, “Sensor Measurement Lists (SenML),” IETF RFC 8428, 2018. https://www.rfc-editor.org/rfc/rfc8428

[2] SQLite, “PRAGMA user_version.” https://www.sqlite.org/pragma.html#pragma_user_version

[3] SQLite, “Appropriate Uses For SQLite.” https://www.sqlite.org/whentouse.html

[4] SQLite, “ALTER TABLE.” https://www.sqlite.org/lang_altertable.html

[5] I. Levkivskyi, J. Lehtosalo, Ł. Langa, “PEP 544: Protocols: Structural subtyping,” 2017. https://peps.python.org/pep-0544/

[6] J. Bezos, “2015 Letter to Shareholders,” Amazon, 2016.

[7] DIAL Control Center repository: svc/app/state.py, svc/app/models.py, svc/app/sensors/manager.py, svc/app/sensors/interface.py, AGENTS.md.